Recurrent Neural Networks

George Michailidis

gmichail@ucla.edu

STAT 426

What is Sequential Data?

- Definition: Data where the order matters. $X = \{x_1, x_2, \ldots, x_T\}$.
- Numerical Time Series: $x_t \in \mathbb{R}^d$.
  - Financial or sales (stock prices, product aggregate sales).
  - Medical vitals (ECG, EEG, Blood Pressure).
  - Data from sensors (Temperature, Vibration).
- Statistical Objective: Estimate the conditional probability of the future:

$$\mathbb{P}(x_t \mid x_{t-1}, x_{t-2}, \ldots, x_1)$$

Text, Audio, Videos as Sequential Data

- Text as Discrete Sequences:
  - Documents are sequences of tokens (chars or words) x_t.
  - Language Modeling: predicting the next token x_t given context.
- Audio as Continuous Signals:
  - Sound is a sequence of amplitude values sampled over time.
  - High Resolution: Standard audio (16kHz) = 16,000 steps/sec.
  - Unlike text (discrete), audio is typically continuous ($x_t \in \mathbb{R}$).
  - *Challenge:* Modeling long-range dependencies due to sequence length.
- Video as High-Dimensional Sequences:
  - A sequence of images (frames): $x_t \in \mathbb{R}^{H \times W \times C}$.
  - Captures temporal dynamics (motion, physics) + spatial features.
  - Seq2Seq Tasks: Video captioning (Input: Video $\rightarrow$ Output: Text).

For this exposition, we will primarily focus on numerical time series data.

Note: The modeling framework presented here is generalizable and can be adapted for other forms of sequential data.

Stationary Sequences - I

The concept of stationarity is fundamental for time series data

Consider a sequence $\{x_1, \cdots, x_t, \cdots\}$ of numerical observations

The sequence is defined to be stationary if

$$\mathbb{P}\big(x_1, \cdots, x_t\big) = \mathbb{P}\big(x_{1+s}, \cdots, x_{t+s}\big), \quad \text{for all} \quad s \in \mathbb{Z} \tag{1}$$

i.e., the probability distribution of a sequence of t observations does not depend on the time origin of the sequence

This is a very strong assumption, because it requires that all aspects of behavior of the sequence are time invariant

It is also a difficult assumption to test empirically

Stationary Sequences - II

A more practical notion is that of weak stationarity

A sequence is weakly stationary if its mean, variance, and covariance are invariant to time shifts

Specifically,

1. $\mathbb{E}(x_t) = \mu$
2. $\text{Var}(x_t) = \sigma^2$ (finite and constant)
3. $\text{Cov}(x_t, x_s) = \gamma(|t - s|)$, for all $t, s \in \mathbb{Z}, t \neq s$ and some function $\gamma(\cdot)$

Hence, the mean and variance do not change over time and the covariance/correlation between two observations only depends on the distance between them

Example: consider a stationary sequence $\{x_t\}$; then,
$\text{Corr}(x_3, x_5) = \text{Corr}(x_{473}, x_{565})$, since both pairs are separated by 95 time units

Stationary Sequences - III

$\gamma(\cdot)$ is called the autocovariance function and $\rho(\cdot)$ the autocorrelation function

Note that

1. $\gamma(h) = \sigma^2 \rho(h), \ h \in \mathbb{Z}$
2. $\gamma(0) = \sigma^2$; $\rho(0) = 1$
3. $\rho(h) = \gamma(h)/\gamma(0)$

Examples of Stationary Sequences

1. White Noise
2. Autoregressive of order p (AR(p))
3. Moving average of order q (MA(q))
4. ARMA(p, q)
5. ARIMA

An Illustrative Simple Time Series Model: AR(1)

It is defined as

$$\big(x_t - \mu\big) = \phi\big(x_{t-1} - \mu\big) + \epsilon_t, \tag{2}$$

for some constant parameters μ, σ_ϵ^2

It can be seen that the sequence $\{x_t\}$ has "memory", in the sense that past values influence the current one

An Illustrative Simple Time Series Model: AR(1)

It is defined as

$$\big(x_t - \mu\big) = \phi\big(x_{t-1} - \mu\big) + \epsilon_t, \tag{2}$$

for some constant parameters μ, σ_ϵ^2

It can be seen that the sequence $\{x_t\}$ has "memory", in the sense that past values influence the current one

Question:
For what values of ϕ is $\{x_t\}$ a weakly stationary sequence?

AR(1) Model with $|\phi| < 1$

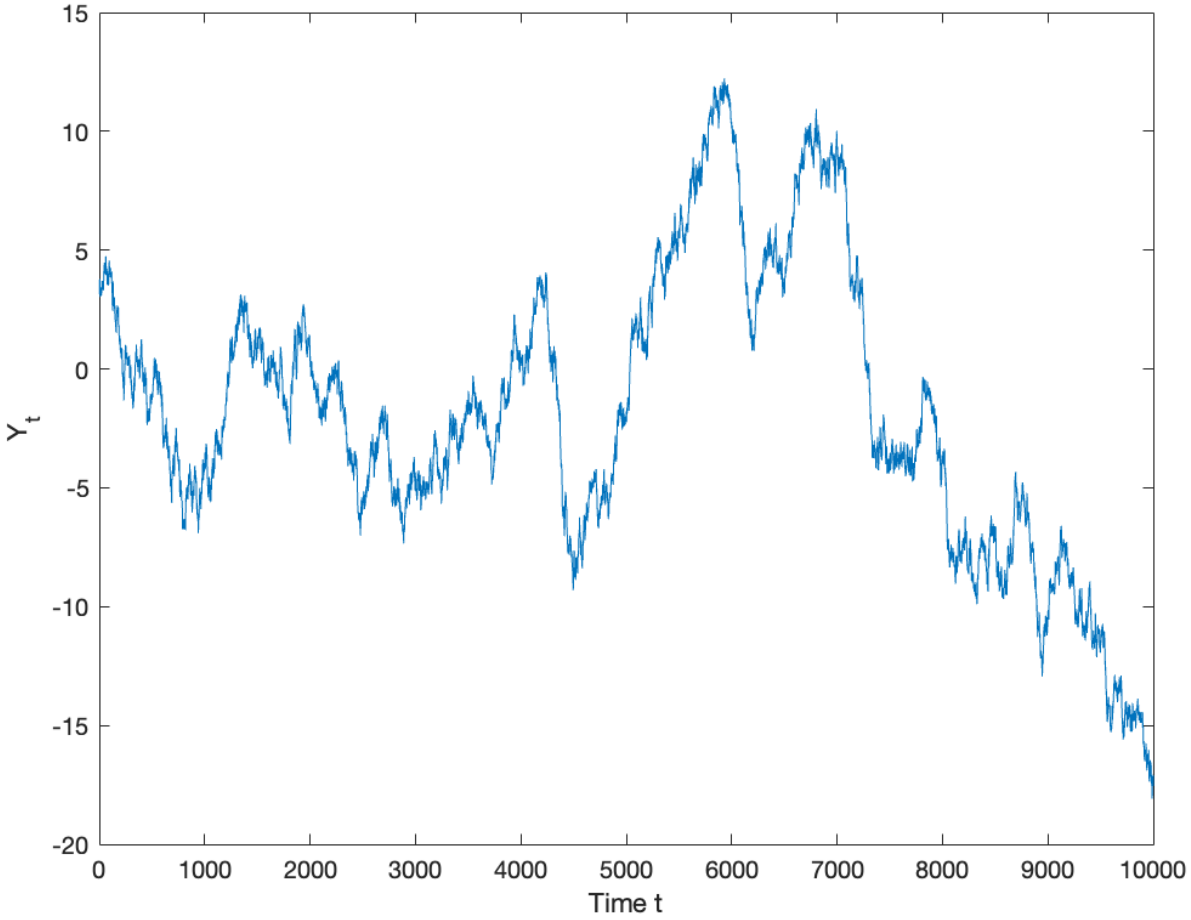


Figure 1: Sample path of AR(1) sequence with $\mu = 0, \sigma_\epsilon = 0.2$ and $X_0 = 3$

AR(1) Model with $|\phi| \geq 1$

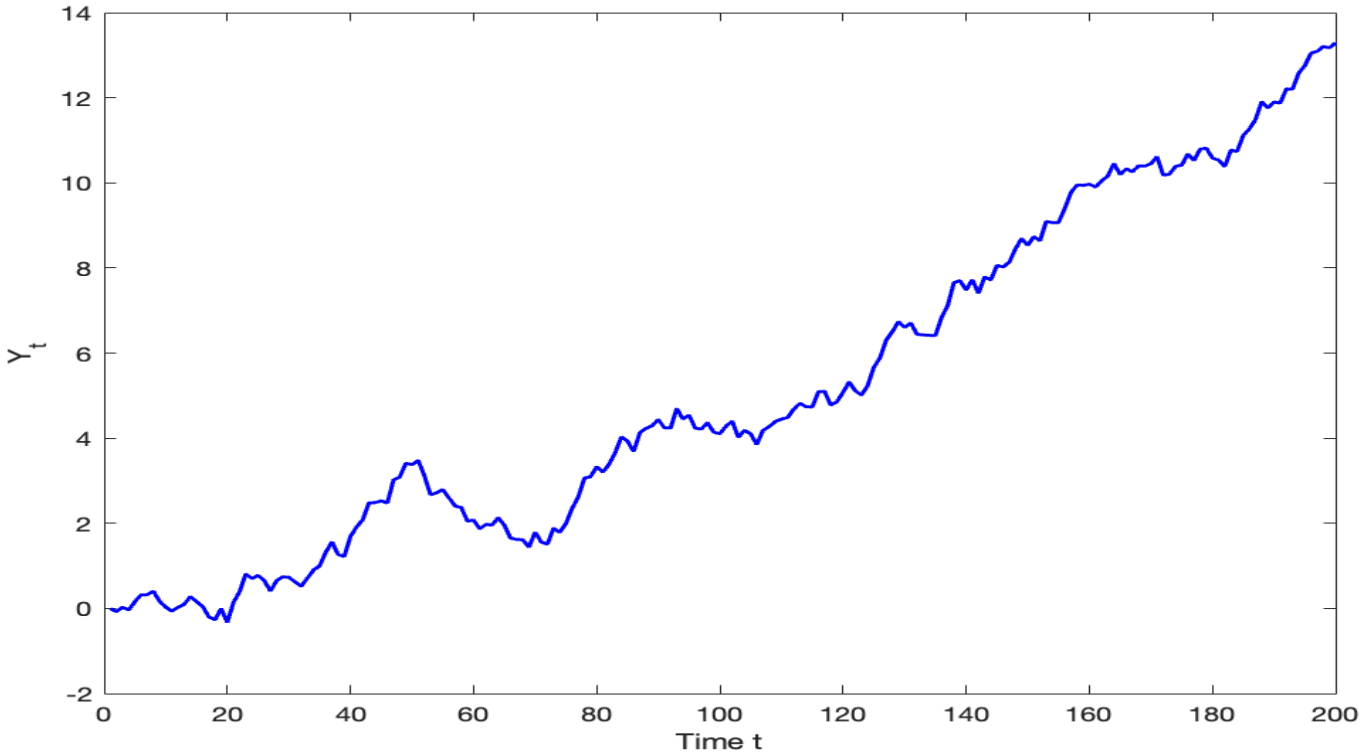


Figure 2: Sample path of AR(1) model with $\phi = 1.01$, $\mu = 0, \sigma_\epsilon = 0.2$ and $X_0 = 0$

The sequence becomes explosive

The AR(p) Model - I

Let $\{\epsilon_t\}$ be an independently and identically distributed sequence with $\mathbb{E}(\epsilon_t) = 0$ and $\text{Var}(\epsilon_t) = \sigma_\epsilon^2$

An autoregressive sequence of order p is defined as

$$\left(x_t - \mu\right) = \sum_{j=1}^{p} \phi_j \left(x_{t-j} - \mu\right) + \epsilon_t, \tag{3}$$

for some constant parameters $\phi_j, \mu, \sigma_\epsilon^2$

Change of notation: from x_t to y_t, in order to put the model in a regression format.

Estimation via Linear Regression

To estimate parameters, rewrite Eq. (3) as a standard linear model:

$$y_t = \mu - \sum_{j=1}^{p} \phi_j \mu + \sum_{j=1}^{p} \phi_j y_{t-j} + \epsilon_t$$

$$y_t = \underbrace{\mu \left(1 - \sum_{j=1}^{p} \phi_j\right)}_{\alpha \text{ (intercept)}} + \sum_{j=1}^{p} \phi_j y_{t-j} + \epsilon_t$$

OLS Procedure:

1. Construct feature vector x_t using lags $y_{t-1}, \ldots, y_{t-p}$ and a bias term (intercept); i.e,

$$y_t = \alpha + \phi_1 x_{1t} + \phi_2 x_{2t} + \cdots + \phi_p x_{pt} + \epsilon_t,$$

   where $x_{1t} = y_{t-1}, \cdots, x_{pt} = y_{t-p}$.
2. Regress current values y_t on x_t to find coefficients $\hat{\alpha}, \hat{\phi}_1, \ldots, \hat{\phi}_p$.
3. Recover μ:

$$\hat{\mu} = \frac{\hat{\alpha}}{1 - \sum_{j=1}^{p} \hat{\phi}_j}$$

AR models and MLPs

The AR(p) model can be formulated as a regression problem, and hence based on Lecture 2, is mathematically equivalent under squared error (ℓ_2) loss to a single-layer Neural Network (MLP) with a linear activation function.

AR(p) Formulation:

$$y_t = \alpha + \sum_{j=1}^{p} \phi_j \underbrace{y_{t-j}}_{\text{input}}$$

Linear MLP Formulation:

$$\hat{y} = b + \sum_{j=1}^{p} w_j x_j$$

Mapping:

- Input layer: Lags $(y_{t-1}, \ldots, y_{t-p})$
- Weights w: AR Coeffs ϕ
- Bias b: Intercept α
- Activation: Identity $f(z) = z$

Limitation: A single linear layer can only model linear dependencies. If the relationship between past and future is complex, this model underfits.

Introducing Nonlinearity: The Standard MLP

To capture complex dynamics, we replace the linear combination with an MLP.

$$\hat{y}_t = f_{\text{MLP}}(y_{t-1}, y_{t-2}, \ldots, y_{t-p}; \theta)$$

- Input: Fixed window of lags $\mathbf{x} = [y_{t-1}, \ldots, y_{t-p}]$.
- Hidden Layer: Creates learned features $h = \sigma(W_1\mathbf{x} + b_1)$.
- Output: Linear projection of features $\hat{y}_t = W_2 h + b_2$.

This is effectively "Nonlinear Autoregression." The model learns its own features rather than relying solely on raw lags.

MLP for Time Series: Pros and Cons

Pros (The Good)

- Universal Approximation: Can model arbitrary non-linear functions (e.g., regime changes, saturation effects).
- Interactions: Automatically learns how lags interact (e.g., "if y_{t-1} is high AND y_{t-2} is low").
- No Assumptions: Does not explicitly assume error normality or strict stationarity (though some form of stability or weak stationarity is often helpful in practice.

Cons (The Bad)

- Fixed Window: Like AR(p), standard MLPs cannot see history beyond lag p.
- Data Hungry: More parameters (weights matrices) require more data to avoid overfitting compared to simple AR.
- Black Box: We lose the interpretability of ϕ coefficients (e.g., "market memory decays by 0.5").

From Point Estimates to Probabilistic Modeling

1. The Limitation of Regression (AR/MLP)
   - Standard autoregressive models minimize the ℓ_2 loss function (squared error - MSE).
   - This implicitly models only the conditional expectation:

$$\hat{x}_t = \mathbb{E}[x_t \mid x_{t-1}, \ldots, x_1]$$

   - Issue: It ignores the shape of the distribution (uncertainty, tails).
2. The Probabilistic Goal
   - We want to model the full conditional distribution:

$$\mathbb{P}(x_t \mid x_{t-1}, \ldots, x_1)$$

   - *Challenge:* The history $\{x_{t-1}, \ldots, x_1\}$ grows indefinitely with t.

The Solution: The Markov Assumption

The Markov Assumption

- First-Order Markov: The future is independent of the past, given the *immediate* present.

$$\mathbb{P}(x_t \mid x_{t-1}, \ldots, x_1) \approx \mathbb{P}(x_t \mid x_{t-1})$$

- k-th Order Markov: The future depends on the last k steps.

$$\mathbb{P}(x_t \mid x_{t-1}, \cdots, x_{t-k}, \cdots, x_1) \approx \mathbb{P}(x_t \mid x_{t-1}, \cdots, x_{t-k})$$

- Approximation: Real-world data is rarely strictly Markovian, but we often pretend it is to make computation tractable.

AR Models and the Markov Property

- AR(1) as a First-Order Markov Process:
  - Equation: $x_t = \phi x_{t-1} + \epsilon_t$ (assuming $\mu = 0$)
  - Relation: The value x_t is completely determined by x_{t-1} (plus noise).
  - Therefore: $\mathbb{P}(x_t \mid x_{t-1}, \ldots, x_1) = \mathbb{P}(x_t \mid x_{t-1})$.

- AR(p) as a p-th Order Markov Process:
  - Equation: $x_t = \sum_{j=1}^{p} \phi_j x_{t-j} + \epsilon_t$ (assuming $\mu = 0$).
  - Relation: We need a history window of size p to define the distribution of x_t.
  - Therefore: $\mathbb{P}(x_t \mid x_{t-1}, \ldots, x_1) = \mathbb{P}(x_t \mid x_{t-1}, \ldots, x_{t-p})$.

Switching Gears: From History to Hidden States

1. The Autoregressive View (What we see)
   - In AR models, we predict the future using past observations.
   - *Assumption:* The history of data $x_{t-1}, \cdots, x_{t-p}$ contains all necessary information.

2. The Real-World Problem (What if data is noisy?)
   - Imagine tracking a car with a noisy GPS.
   - The GPS readings (x) are erratic, but the true car position (h) moves smoothly.
   - If we rely only on past noisy GPS points (AR), we predict erratic movement.

3. The State Space Solution
   - Instead of modeling $x_{t-1} \to x_t$, we model a Hidden State h_t (the true car position).
   - Dynamics: The true state evolves smoothly ($h_{t-1} \to h_t$).
   - Key message: We see a noisy version of the state ($h_t \to x_t$).

What is a State Space Model (SSM)?

Core Idea:

- We assume the observations y_t are generated by an underlying, unobserved (latent) sequence h_t.
- The latent state h_t evolves over time and captures the "dynamics" of the system.

Two Key Components:

1. Transition Equation (Dynamics): Describes how the hidden state evolves from $t-1$ to t.

$$h_t = \text{Dynamics}(h_{t-1}) + \text{Noise}$$

2. Observation Equation (Measurement): Describes how the observation is derived from the current state.

$$y_t = \text{Measurement}(h_t) + \text{Noise}$$

Linear State Space Models (LSSM)

A Generalized Framework: In the general case, we model a hidden state $\mathbf{h}_t$ driven by an external input $\mathbf{u}_t$ and observed as $\mathbf{y}_t$.

$$\mathbf{h}_t = \mathbf{A}\mathbf{h}_{t-1} + \mathbf{B}\mathbf{u}_t + \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$
$$\mathbf{y}_t = \mathbf{C}\mathbf{h}_t + \boldsymbol{\eta}_t, \quad \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$$

Notation Mapping:

- $\mathbf{h}_t$: Hidden State (The memory).
- $\mathbf{y}_t$: Observation (the data we see).
- $\mathbf{u}_t$: Input/Control (The external force).

Note: In Time Series (AR), we often assume $\mathbf{B} = \mathbf{0}$ (no external input). In Deep Learning (RNNs), this $\mathbf{u}_t$ becomes our data input $\mathbf{x}_t$.

Remarks

Another Key Distinction: Multivariate Capability

- Previous AR(p): Focused on *univariate* scalar data ($x_t \in \mathbb{R}$).
- LSSM: Seamlessly handles multivariate time series ($\mathbf{y}_t \in \mathbb{R}^m$).
  - Example: Tracking Position (ℓ) and Velocity (v) simultaneously.
  - The matrices $\mathbf{Q}$ and $\mathbf{R}$ are covariance matrices, capturing correlations between different variables.

Famous Example: The Kalman Filter is the exact solution for estimating the vector $\mathbf{h}_t$ in this model.

Example: AR(1) as a Linear SSM

Recall the AR(1) process:

$$x_t = \phi x_{t-1} + \epsilon_t \quad (\mu = 0)$$

We can map this directly to the SSM framework by setting the state h_t to be the value x_t itself.

SSM Mapping:

- State (Transition): $h_t = \phi h_{t-1} + \epsilon_t$, with $y_t = 0$ for all t.
- Observation: $y_t = 1 \cdot h_t$ (Perfect observation, no measurement noise)

Markov Property: Since h_t depends only on h_{t-1}, the AR(1) process is strictly First-Order Markovian.

Example: AR(p) via State Augmentation

Problem: AR(p) depends on p past values, so it appears non-Markovian (depends on history).

$$x_t = \sum_{j=1}^{p} \phi_j x_{t-j} + \epsilon_t$$

The State Vector: We stack the p lags into a single hidden state vector $\mathbf{h}_t$; note $\mathbf{u}_t = \mathbf{0}$ for all t:

$$\mathbf{h}_t = \begin{bmatrix} x_t \\ x_{t-1} \\ \vdots \\ x_{t-p+1} \end{bmatrix}$$

Restoring the Markov Property: The transition $\mathbf{h}_t = \mathbf{A}\mathbf{h}_{t-1} + \boldsymbol{\epsilon}_t$ is now first-order Markovian. The matrix $\mathbf{A}$ simply shifts values down and computes the new x_t at the top.

From Linear to Nonlinear SSMs

Real-world dynamics are rarely perfectly linear. We generalize the transition and emission matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ to non-linear functions (f, g).

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{u}_t) + \boldsymbol{\epsilon}_t \quad \text{(Nonlinear Transition)}$$
$$\mathbf{y}_t = g(\mathbf{h}_t) + \boldsymbol{\eta}_t \quad \text{(Nonlinear Emission)}$$

Key Characteristics:

- $f(\cdot)$ captures complex temporal evolution (e.g., saturation, physics).
- $g(\cdot)$ maps the latent state to the observed data space.
- Still inherently stochastic (due to $\boldsymbol{\epsilon}_t, \boldsymbol{\eta}_t$).

Connecting Nonlinear SSMs to RNNs

The Bridge: A Recurrent Neural Network (RNN) is a Nonlinear SSM with two simplifications.

First, we switch notation: Control Input $\mathbf{u}_t \to$ *Data Input* $\mathbf{x}_t$.

1. Deterministic Dynamics: Process noise is removed. The state evolves deterministically.
$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$
2. Neural Parameterization: The functions are approximated by neural networks (weights $\mathbf{W}, \mathbf{U}$).
$$\mathbf{h}_t = \sigma(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t + \mathbf{b}), \qquad \mathbf{y}_t = g(\mathbf{h}_t)$$

RNNs replace explicit probabilistic state-space modeling with deterministic, data-driven dynamics learned via backpropagation through time (unless explicitly augmented with stochastic layers or variational objectives).

The Vanilla RNN Cell: Processing Numerical Data

We process the time series one step at a time.

1. Input ($\mathbf{x}_t$): The numerical value/vector at time t.
2. Hidden State ($\mathbf{h}_{t-1}$): Memory from previous steps.
3. Output ($\mathbf{y}_t$): The prediction at time t.

The Update Equation:

$$\mathbf{h}_t = \tanh\left(\underbrace{\mathbf{W}_{hh}\mathbf{h}_{t-1}}_{\text{Past info}} + \underbrace{\mathbf{W}_{xh}\mathbf{x}_t}_{\text{New input}} + \mathbf{b}\right)$$

The RNN learns a nonlinear transformation (via tanh*) that combines the "memory" of past data with the current values.*

Digression: the Hyperbolic Tangent Function (Tanh)

Definition:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Visual Properties:

- Range: Outputs are strictly bounded between [-1, 1].
- Zero-Centered: Unlike Sigmoid (which is [0, 1]), Tanh crosses through the origin. This helps gradients flow more efficiently during training.
- Shape: S-curve that "saturates" for large positive or negative inputs.

Why Tanh and not ReLU in Vanilla RNNs?

The Problem: Exploding Activations

- In an RNN, we multiply by the *same* weight matrix $\mathbf{W}$ at every time step:

$$\mathbf{h}_t = \sigma(\mathbf{W}\mathbf{h}_{t-1} + \cdots)$$

- ReLU is Unbounded $[0, \infty)$: If the weights are even slightly larger than 1, the values in $\mathbf{h}_t$ can grow exponentially (e.g., $1.1^{100} \approx 13,780$). The state eventually explodes to NaN (Not a Number).
  Recall exploding time series data generated by an AR(1) model with $|\phi| > 1$.
- Analogous problems appear in back-propagating gradients.

The Solution: Boundedness

- Tanh is Bounded $[-1, 1]$: It "squashes" the hidden state at every single step.
- No matter how large the weights are, $\mathbf{h}_t$ can never exceed 1 or drop below -1, ensuring Forward Pass Stability.

Digression: RNN Formulation for Univariate Time Series Data

The Scenario: We want to forecast a single variable y_t (e.g., Stock Price) using *only* its own history. We have no external features ($\mathbf{u}_t$ or "covariates").

The Simplification: The "External Input" slot is empty. We fill the network's input vector x_t entirely with the previous observation.

The Input Mapping

$$\text{RNN Input } x_t := y_{t-1}$$

The Equation becomes Purely Autoregressive:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}y_{t-1} + b)$$

In this setting, the RNN implements a nonlinear autoregressive model with implicit, potentially infinite memory encoded in the hidden state h_t.

Digression: Generalizing to Multivariate Time Series

The Scenario: We want to forecast M variables simultaneously (e.g., Temperature, Humidity, and Pressure).

$$\mathbf{y}_t \in \mathbb{R}^M$$

The Generalization: Instead of feeding a single scalar number as input, we feed the entire vector of previous observations.

The Vectorized Input

$$\text{RNN Input } \mathbf{x}_t := \mathbf{y}_{t-1} = \begin{bmatrix} y_{t-1}^{(1)} \\ \vdots \\ y_{t-1}^{(M)} \end{bmatrix}$$

Why is this powerful? The hidden state h_t learns not just the history of each variable, but their cross-variable temporal dependencies:

- *Example:* "If Humidity rose yesterday ($y_{t-1}^{(2)} \uparrow$), then Temperature drops today ($y_t^{(1)} \downarrow$)."
- The weight matrix W_{xh} automatically captures these interactions.

Recap: Constructing the RNN Input Vector (x_t)

The Generic "Input" ($\mathbf{x}_t$): In the standard RNN equation

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t),$$

the variable $\mathbf{x}_t$ is just a container. It represents everything available to the model at time t.

The "Concatenation" Strategy: We construct $\mathbf{x}_t$ by stacking two distinct types of information:

$$\mathbf{x}_t = \text{Concat}(\underbrace{\mathbf{u}_t}_{\text{External Data}}, \underbrace{\mathbf{y}_{t-1}}_{\text{Lagged History}})$$

- $\mathbf{u}_t$ (Forcing/Covariates): Things we know about the present (e.g., "Is it a holiday?").
- $\mathbf{y}_{t-1}$ (Feedback): What just happened (e.g., "Yesterday's price").

Scenario Mapping: Pure vs. Mixed Problems

Depending on the data available, we adapt the input vector $\mathbf{x}_t$:

Case A: Pure Time Series

- We have no external info.
- We rely solely on history.
- The "External" slot is empty.

$$\mathbf{x}_t = [\mathbf{y}_{t-1}]$$

(Equivalent to an AR model)

Case B: Mixed (with Covariates)

- We have history plus external drivers.
- e.g., Sales (y) affected by Promotions (u).

$$\mathbf{x}_t = [\mathbf{u}_t, \mathbf{y}_{t-1}]$$

(Equivalent to AR-X / ARMAX)

Key Takeaway: The RNN architecture does not change. We just change what we feed into the vector $\mathbf{x}_t$.

RNN Input–Output Structures

Why Variants?

- Different sequence problems require different input–output relationships.
- RNNs provide a flexible template that can be adapted by changing:
  - how many inputs are observed, and
  - how many outputs are produced.

RNN Input–Output Structures - II

Common RNN Input–Output Patterns

- One-to-One
  - Single input → single output
  - (Equivalent to standard regression; no sequence modeling)
- One-to-Many
  - Single input → output sequence
  - Used for generation tasks (e.g., forecasting, text generation)
- Many-to-One
  - T=Multivariate time series input input → single label
  - Used for classification, prediction tasks
- Many-to-Many
  - Input sequence → output sequence
  - Used for sequence transformation tasks (e.g., translation, labeling)

Next, we examine each structure and the types of problems it is designed to solve.

1. One-to-One: The Non-Recurrent Baseline

Structure:

- Input: A vector $\mathbf{x}_t$.
- Output: A vector $\mathbf{y}_t$.
- Crucial Distinction: This is NOT an RNN. It is a standard Feedforward Network (MLP).

Where is $\mathbf{h}_t$?

- In a true RNN: $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$.
- In One-to-One: The link to the past is broken ($\mathbf{W}_{hh} = \mathbf{0}$).

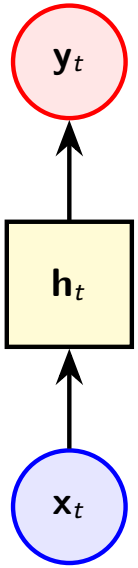
$$\mathbf{h}_t = f(\mathbf{x}_t)$$

- The hidden state $\mathbf{h}_t$ exists only to process the *current* input, not to store memory.

Numerical Context:

- This is standard Cross-Sectional Regression.
- $$\mathbf{y}_t = \text{MLP}(\mathbf{x}_t)$$

1. One-to-One: The Non-Recurrent Baseline



Characteristics:

- Standard Feedforward Network (MLP).
- **No Memory:** $\mathbf{h}_t$ depends only on $\mathbf{x}_t$.
- $\mathbf{W}_{hh} = \mathbf{0}$.

Example:

- Image Classification (Image $\rightarrow$ Label).

2. One-to-Many: Trajectory Projection

The Scenario (Pure Autoregression)

- Assumption: We have no future external drivers ($\mathbf{x}_t = \mathbf{0}$ for $t > 1$).
- Input: A single initial seed $\mathbf{y}_0$ (e.g., the last known value).
- Output: Forecasted trajectory $\hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \ldots, \hat{\mathbf{y}}_T$.

RNN Implementation Since the external input slot is empty, we fill it with the Feedback Loop.

1. Initialization ($t = 1$): Feed the initial seed $\mathbf{y}_0$ into the input matrix $\mathbf{W}_{xh}$.

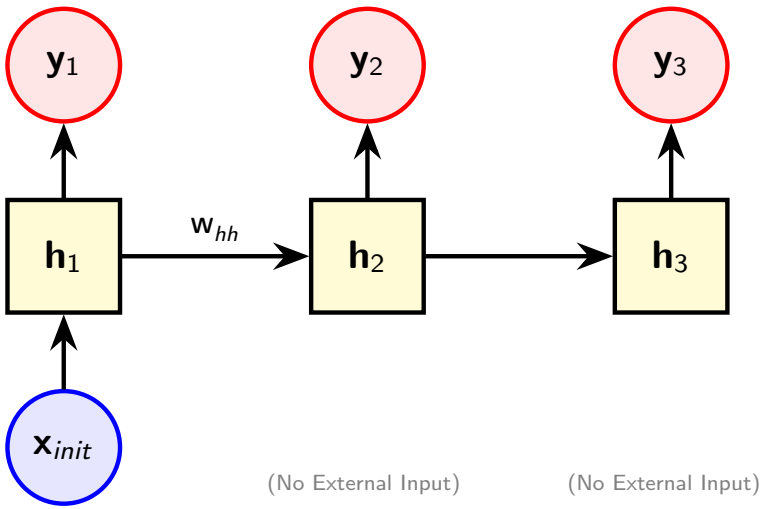
$$\mathbf{h}_1 = \tanh(\mathbf{W}_{hh}\mathbf{h}_0 + \mathbf{W}_{xh}\mathbf{y}_0)$$

2. Autonomous Evolution ($t > 1$): Feed the Predicted Feedback $\hat{\mathbf{y}}_{t-1}$ into the input matrix.

$$\hat{\mathbf{y}}_{t-1} = g(\mathbf{h}_{t-1}), \qquad \mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\hat{\mathbf{y}}_{t-1})$$

The "Input" to the network becomes its own previous "Output."

2. One-to-Many: Trajectory Projection



- **Input:** Single seed ($\mathbf{x}_{init}$) or initial state.
- **Output:** A sequence of values.
- **Key Mechanism:** Pure Autoregression (the output drives the next step).
- **Example:** Music Generation, Trajectory Forecasting.

Teacher Forcing vs. Autoregression

Goal: Compute the hidden state $\mathbf{h}_t$ and prediction $\hat{\mathbf{y}}_t$.

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$$

The Input Structure: Recall that $\mathbf{x}_t$ is a concatenation of External Data ($\mathbf{u}_t$) and Feedback ($\mathbf{f}_t$):

$$\mathbf{x}_t = [\mathbf{u}_t, \mathbf{f}_t]$$

A. Training (Teacher Forcing)

- We define the feedback $\mathbf{f}_t$ as the Ground Truth:

$$\mathbf{f}_t := \mathbf{y}_{t-1}$$

- Resulting Input:

$$\mathbf{x}_t = [\mathbf{u}_t, \mathbf{y}_{t-1}]$$

- We "break" the causal chain. The model's previous error is ignored; we manually reset the path using the true history.

B. Testing (Inference)

- We define the feedback $\mathbf{f}_t$ as the Model Prediction:

$$\mathbf{f}_t := \hat{\mathbf{y}}_{t-1}$$

- Resulting Input:

$$\mathbf{x}_t = [\mathbf{u}_t, \hat{\mathbf{y}}_{t-1}]$$

- The system is now Autoregressive. Errors at $t-1$ become inputs at t, leading to "Error Drift."

3. Many-to-One: Sequence Aggregation

The Scenario (Classification & Regression)

- Input Sequence: A full sequence of vectors $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$.
- Output: A single label or value $\mathbf{y}$ at the very end.
- Use Cases: Sentiment Analysis (Text $\rightarrow$ Rating), Time Series Classification (ECG $\rightarrow$ Diagnosis).

RNN Implementation The RNN processes the entire sequence, accumulating information into the hidden state, but produces an output only at the final step.

1. The Accumulation Phase ($t = 1 \ldots T$): Update the hidden state repeatedly. We ignore any intermediate outputs.

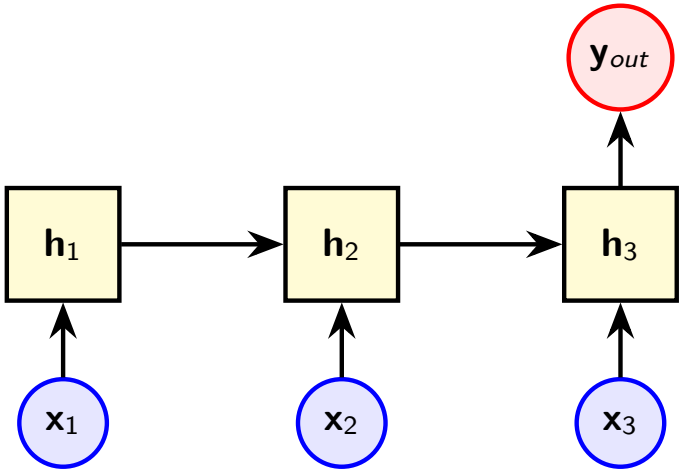
$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$$

2. The Readout Phase ($t = T$): Once the sequence is finished, the final state $\mathbf{h}_T$ summarizes the entire history. We use it to predict $\mathbf{y}$.

$$\hat{\mathbf{y}} = g(\mathbf{W}_{hy}\mathbf{h}_T + \mathbf{b}_y)$$

The final hidden state $\mathbf{h}_T$ acts as a "Feature Vector" representing the whole sequence.

3. Many-to-One: Sequence Aggregation



- **Input:** A sequence of features.
- **Output:** A single decision at the end.
- **Key Mechanism:** Information accumulation. Intermediate outputs are ignored.
- **Example:** Sentiment Analysis, Time Series Classification.

4. Many-to-Many: Sequence Transduction

The Scenario (Full Dynamic Modeling)

- Input Sequence: A series of external drivers $\mathbf{u}_1, \ldots, \mathbf{u}_T$.
- Output Sequence: A corresponding sequence of targets $\mathbf{y}_1, \ldots, \mathbf{y}_T$.
- Goal: Predict $\mathbf{y}_t$ using both the current driver $\mathbf{u}_t$ and past information.

RNN Implementation We use a concatenated input that combines new data and feedback.

1. The Mechanism: At each time step t, the RNN consumes the external input and feedback from the previous step:

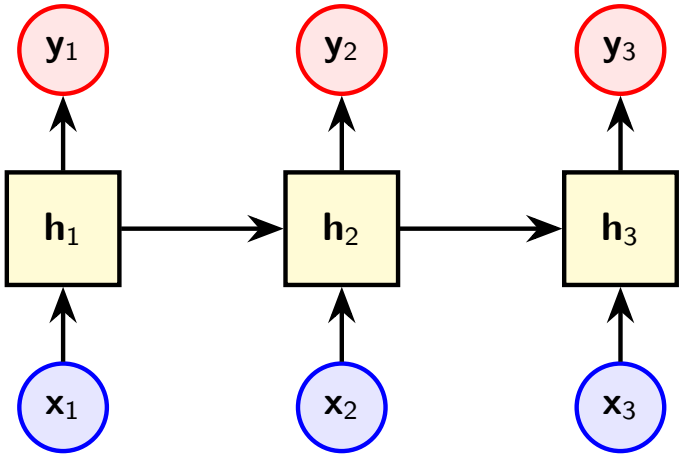
$$\mathbf{x}_t = [\ \underbrace{\mathbf{u}_t}_{\text{External Driver}}\ ,\ \underbrace{\mathbf{f}_t}_{\text{Previous Output}}\ ] \quad \longrightarrow \quad \mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$$

2. Synchronized Output: An output is produced at every time step:

$$\hat{\mathbf{y}}_t = g(\mathbf{h}_t)$$

This general setting is commonly referred to as Dynamic Regression, Nonlinear System Identification, or Nonlinear Autoregressive with External Inputs (NARX) modeling.

4. Many-to-Many: Synchronized Transduction



- **Input:** A sequence of vectors.
- **Output:** An aligned sequence of vectors ($T_{in} = T_{out}$).
- **Key Mechanism:** Immediate response to current input + memory.
- **Example:** Frame-by-frame Video Classification, Named Entity Recognition.

The Limitation of Synchronous Frameworks

Current Assumption: Lock-Step Alignment The architectures discussed so far (One-to-Many, Many-to-Many) are synchronous.

- constraint: The input sequence length T_{in} must equal the output sequence length T_{out}.
- mapping: Every input $\mathbf{u}_t$ triggers an immediate update and potential output $\mathbf{y}_t$.
- use case: Ideal for *online* monitoring, filtering, or frame-by-frame labeling.

The Asynchronous Challenge Many complex sequence problems are asynchronous ($T_{in} \neq T_{out}$):

- Language: Translation (10 words in English $\rightarrow$ 14 words in German).
- Time Series: "Read the last 24 hours ($\mathbf{u}_{1\ldots24}$), then predict the next 5 hours ($\mathbf{y}_{1\ldots5}$)."

To handle these decoupled sequences, we will later introduce Encoder-Decoder (Seq2Seq) architectures, which separate the "Reading" phase from the "Generating" phase.

Comparison: Recursive RNN vs. Seq2Seq Preview

1. Standard Recursive RNN (The "Rolling" Approach) We use a single model ($\mathbf{W}_{hh}$) for both processing history and generating forecasts.

- Process: Read $\mathbf{y}_t \rightarrow$ Predict $\mathbf{y}_{t+1} \rightarrow$ Feed back $\hat{\mathbf{y}}_{t+1} \ldots$
- Limitation: The model cannot "pause and think." It must produce an output at every single step. It is Synchronous.

---

2. Sequence-to-Sequence Approach) We split the problem into two distinct phases using two separate networks.

Phase A: The Encoder

- Reads the entire history window (e.g., past 24 hours).
- *Output:* Compresses everything into a single Context Vector $\mathbf{c}$.
- No predictions are made yet.

Phase B: The Decoder

- Takes $\mathbf{c}$ as its initial seed.
- Generates the future sequence (e.g., next 5 hours).
- *Benefit:* Decouples input length from output horizon.

Comparison: Recursive RNN vs. Seq2Seq Preview

1. Standard Recursive RNN (The "Rolling" Approach) We use a single model (W_{hh}) for both processing history and generating forecasts.

- Process: Read $y_t \rightarrow$ Predict $y_{t+1} \rightarrow$ Feed back $\hat{y}_{t+1} \ldots$
- Limitation: The model cannot "pause and think." It must produce an output at every single step. It is Synchronous.

---

2. Sequence-to-Sequence Approach) We split the problem into two distinct phases using two separate networks.

Phase A: The Encoder

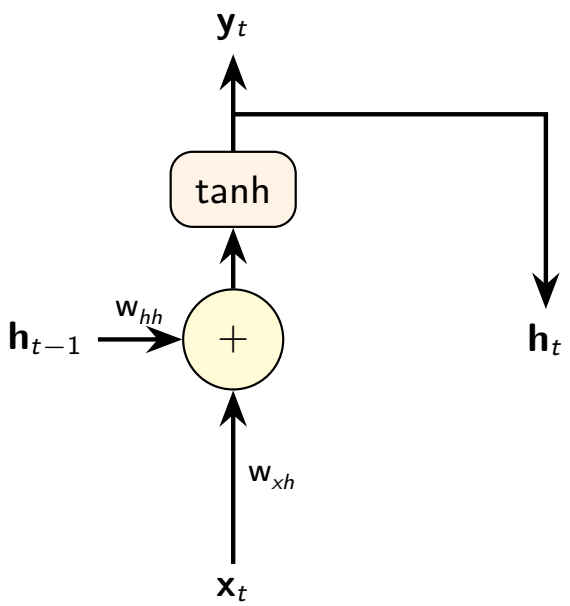
- Reads the entire history window (e.g., past 24 hours).
- *Output:* Compresses everything into a single Context Vector $\mathbf{c}$.
- No predictions are made yet.

Phase B: The Decoder

- Takes $\mathbf{c}$ as its initial seed.
- Generates the future sequence (e.g., next 5 hours).
- *Benefit:* Decouples input length from output horizon.

Review: The Standard RNN Cell (Vanilla)

The Core Mechanism The standard RNN cell is a simple feedback loop. At each time step t, it merges the Present ($\mathbf{x}_t$) with the Past ($\mathbf{h}_{t-1}$) to create the Future ($\mathbf{h}_t$).



1. Update Hidden State:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

2. Compute Output:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

The Structural Flaw:

- The gradient must flow through the same $\mathbf{W}_{hh}$ matrix T times.
- **Result:** Gradients explode (NaN) or vanish (0) for $T > 10$.
- *The memory is rewritten, not maintained.*

The Structural Flaw: Exploding & Vanishing Gradients

The Mathematical Root Cause To update the weights $\mathbf{W}_{hh}$, we propagate the error in $\mathcal{L}$ from the end of the sequence ($t = T$) back to the beginning ($t = 1$).

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \cdot \underbrace{\prod_{t=2}^{T} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}}_{\text{The Chain Rule}}$$

The "Power" Law: Recall that $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \dots)$. The derivative of the state with respect to the past is roughly the weight matrix itself:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \approx \mathbf{W}_{hh}$$

Therefore, the total gradient is proportional to the matrix power:

$$\text{Total Gradient} \propto (\mathbf{W}_{hh})^T$$

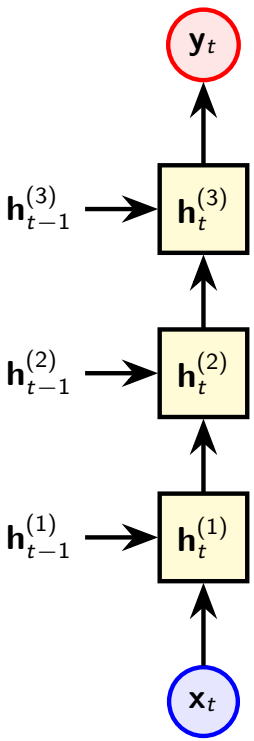
Two Destinies (Eigenvalues λ): Like multiplying a number x by itself 100 times:

- **Exploding ($|\lambda| > 1$):** $1.1^{100} \approx 13,780$.
  - Weights update too much.
  - Loss becomes `NaN`.
  - **Fix:** Gradient Clipping.
- **Vanishing ($|\lambda| < 1$):** $0.9^{100} \approx 0.00002$.
  - Gradient becomes zero.
  - Model "forgets" the start.
  - **Fix:** LSTM / GRU / ResNets.

Deep RNNs: Adding Depth in Space

Two Dimensions of Depth

- Deep in Time: Every RNN is effectively a T-layer network when unrolled. This handles *temporal dependencies*.
- Deep in Space (Stacked RNNs): We stack multiple RNN instances on top of each other. This handles *hierarchical representation*.



The Mechanism: The hidden state of layer $l-1$ becomes the *input* to layer l.

$$\mathbf{h}_t^{(l)} = \tanh(\underbrace{\mathbf{W}_{hh}^{(l)}\mathbf{h}_{t-1}^{(l)}}_{\text{Time Flow}} + \underbrace{\mathbf{W}_{xh}^{(l)}\mathbf{h}_t^{(l-1)}}_{\text{Layer Flow}})$$

Why Stack Layers?

- **Layer 1:** Extracts raw patterns (e.g., sudden spikes).
- **Layer 2:** Composes patterns (e.g., "rising trend").
- **Layer 3:** High-level logic (e.g., "market recovery").

1. LSTM Motivation: Separating Memory from State

The Problem with Standard RNNs

- In a vanilla RNN, the hidden state $\mathbf{h}_t$ serves two conflicting purposes:
  1. Memory: Preserving long-term context (e.g., a yearly seasonal trend).
  2. Processing: Reacting to immediate inputs (e.g., a sudden price spike today).
- Result: Long-term information is easily attenuated as gradients propagate through time.

The LSTM Solution: Dual States

We split the state into two distinct vectors:

1. Cell State ($\mathbf{C}_t$):
   - The "Internal" memory.
   - Acts like a "conveyor belt"; carries long-term trends with minimal modification.
2. Hidden State ($\mathbf{H}_t$):
   - The "External" output.
   - A filtered version of the memory (e.g., "Buy/Sell" signal for financial assets), exposed to the next layer.

2. The Control Mechanism: Three Gates

To control the flow of information into $\mathbf{C}_t$, we use "Gates." A gate is a vector of values in $(0, 1)$ generated by a Sigmoid function.

Input Gate ($\mathbf{I}_t$): How much new info to write?

$$\mathbf{I}_t = \sigma(\mathbf{W}_{xi}\mathbf{x}_t + \mathbf{W}_{hi}\mathbf{H}_{t-1} + \mathbf{b}_i)$$

Forget Gate ($\mathbf{F}_t$): How much old info to keep?

$$\mathbf{F}_t = \sigma(\mathbf{W}_{xf}\mathbf{x}_t + \mathbf{W}_{hf}\mathbf{H}_{t-1} + \mathbf{b}_f)$$

Output Gate ($\mathbf{O}_t$): How much info to reveal?

$$\mathbf{O}_t = \sigma(\mathbf{W}_{xo}\mathbf{x}_t + \mathbf{W}_{ho}\mathbf{H}_{t-1} + \mathbf{b}_o)$$

Notation: σ is the element-wise sigmoid function. All $\mathbf{W}$ are learned weight matrices.

Each gate acts as a soft switch, learned from data.

3. Updating the Memory Cell ($\mathbf{C}_t$)

The core of the LSTM is the update of the Cell State $\mathbf{C}_t$. This happens in two steps:

Step A: Create Candidate Data We calculate a "Candidate" memory $\tilde{\mathbf{C}}_t$ using a tanh activation (similar to a vanilla RNN).

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_{xc}\mathbf{x}_t + \mathbf{W}_{hc}\mathbf{H}_{t-1} + \mathbf{b}_c)$$

Step B: The Flow Control (Forget & Add) We combine the old memory with the new candidate using the gates:

$$\mathbf{C}_t = \underbrace{\mathbf{F}_t \odot \mathbf{C}_{t-1}}_{\text{Keep Past}} + \underbrace{\mathbf{I}_t \odot \tilde{\mathbf{C}}_t}_{\text{Write New}}$$

Key Insight: If $\mathbf{F}_t \approx 1$ and $\mathbf{I}_t \approx 0$, the cell acts as an identity function, preserving gradients perfectly over long distances.

4. Generating the Hidden State ($\mathbf{H}_t$)

The final step is to determine what the network actually "sees" at this time step.

- The Cell State $\mathbf{C}_t$ stores the accumulated internal memory.
- We may not want to output all of it right now.

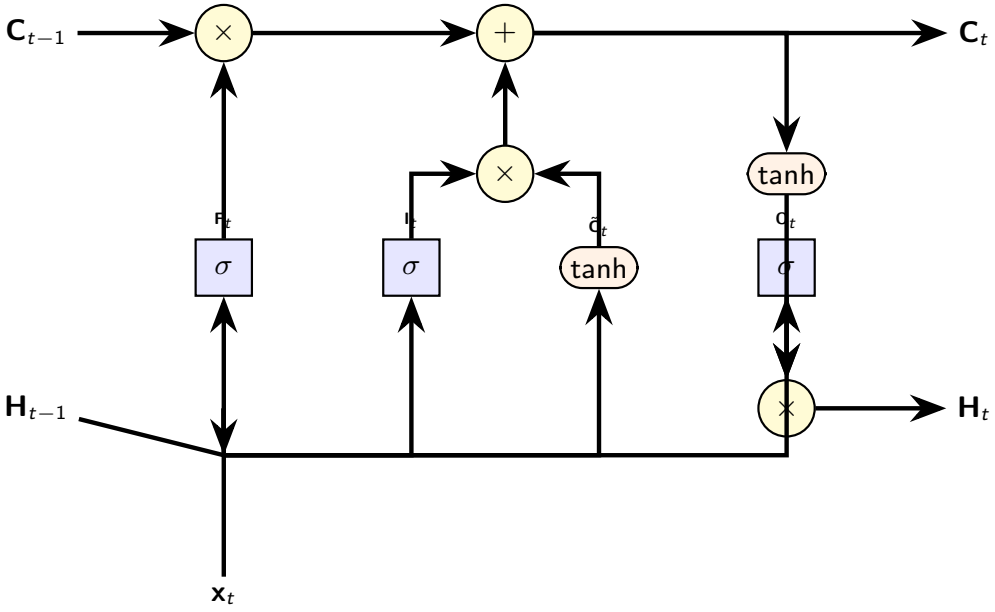
The Output Equation

We pass the cell state through a tanh (to squash values to $[-1, 1]$) and filter it with the Output Gate $\mathbf{O}_t$:

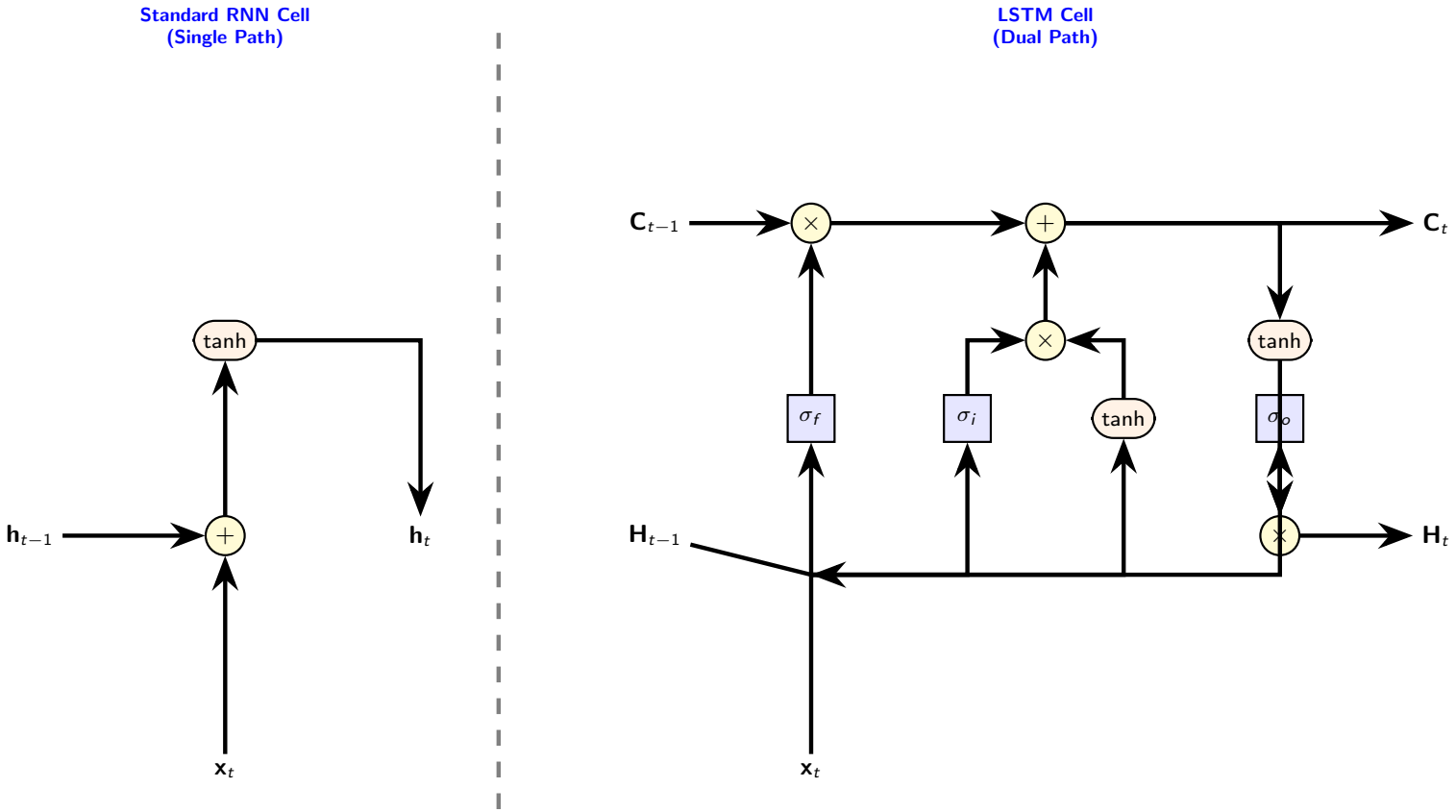
$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t)$$

- $\mathbf{H}_t$ is then passed to:
  1. The next time step (as $\mathbf{H}_{t-1}$).
  2. The output layer (to predict $\hat{\mathbf{y}}_t$).

5. Visual Summary of the LSTM Cell



Visualizing the Difference: Vanilla RNN vs. LSTM



Note: This diagram shows the internal state update. The prediction $\mathbf{y}_t$ is computed externally from $\mathbf{h}_t$, as shown in the next slide.

Structural Complexity & State

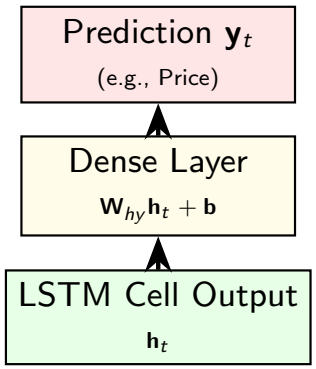
1. The Standard RNN (Left Panel)
   - **Single Track:** There is only one hidden state pathway ($\mathbf{h}_{t-1} \rightarrow \mathbf{h}_t$).
   - **Forced Update:** At every step, the old state is forced to merge with the new input and pass through a tanh squash function.
   - **Consequence:** It is very difficult for information to survive unmodified for many time steps. The "memory" is volatile.

2. The LSTM (Right Panel)
   - **Dual Tracks:** We introduce a second pathway, the Cell State ($\mathbf{C}_t$).
   - **The "Superhighway":** Notice the straight line across the top ($\mathbf{C}_{t-1} \rightarrow \mathbf{C}_t$). Information can flow along this path with only linear interactions (multiplication and addition), avoiding the constant squashing of the tanh function.
   - **Consequence:** The network can choose to preserve long-term memories in $\mathbf{C}_t$ while using $\mathbf{h}_t$ for immediate processing.

Where is the Prediction ($\mathbf{y}_t$)?

The diagram ends at $\mathbf{H}_t$. How do we actually predict $\mathbf{y}_t$?

The "Head" of the Network The LSTM cell is just a feature extractor. To get a prediction, we attach a standard Dense (Linear) layer on top of the hidden state output $\mathbf{H}_t$.



The Sequence of Events:

1. Update State: The LSTM updates $\mathbf{C}_{t-1} \rightarrow \mathbf{C}_t$ and computes $\mathbf{H}_t$.
2. Project: We project $\mathbf{h}_t$ to the output dimension (e.g., 1 value for stock price).
3. Loop (Inference Only): We take the predicted $\mathbf{y}_t$ and feed it back as $\mathbf{x}_{t+1}$ for the next step.

1. Gated Recurrent Units (GRU): Streamlining the LSTM

The Complexity of LSTM

While effective, the LSTM architecture is computationally heavy:

- **Dual States:** Maintains both $\mathbf{C}_t$ and $\mathbf{H}_t$.
- **Three Gates:** Input, Forget, and Output gates.
- **Parameters:** Requires 4 sets of weight matrices.

The GRU Alternative (Cho et al., 2014)

The GRU simplifies the design without sacrificing the ability to capture long-term dependencies.

- **Merged State:** Only a single hidden state $\mathbf{h}_t$ (no separate Cell State).
- **Two Gates:** Combines "Input" and "Forget" concepts into a **Reset** and **Update** gate.
- **Efficiency:** Fewer parameters, faster training, often similar performance.

2. The Control Mechanism: Reset & Update

Just like the LSTM, the GRU uses sigmoid ($\sigma \in (0, 1)$) gates to control information flow.

- **Reset Gate ($\mathbf{R}_t$):** Decides how much of the *past* state should be ignored when computing the new candidate.

$$\mathbf{R}_t = \sigma(\mathbf{W}_{xr}\mathbf{x}_t + \mathbf{W}_{hr}\mathbf{h}_{t-1} + \mathbf{b}_r)$$

- **Update Gate ($\mathbf{Z}_t$):** Decides whether to update the hidden state or keep the old one (acting as both "Input" and "Forget").

$$\mathbf{Z}_t = \sigma(\mathbf{W}_{xz}\mathbf{x}_t + \mathbf{W}_{hz}\mathbf{h}_{t-1} + \mathbf{b}_z)$$

Note: All $\mathbf{W}$ *matrices are learned parameters.*

3. The Candidate Hidden State ($\tilde{\mathbf{h}}_t$)

Before updating the final state, the GRU proposes a "Candidate" state $\tilde{\mathbf{h}}_t$.

The Mechanism:

- It integrates the input $\mathbf{x}_t$ with the previous state $\mathbf{h}_{t-1}$.
- **Crucial Difference:** The previous state is filtered by the Reset Gate $\mathbf{R}_t$ *before* being used.

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}(\mathbf{R}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)$$

Interpretation:

- If $\mathbf{R}_t \approx \mathbf{0}$: The unit acts as if it is reading the first symbol of a sequence (forgetting history).
- If $\mathbf{R}_t \approx \mathbf{1}$: It acts like a standard RNN.

4. The Hidden State Update ($\mathbf{h}_t$)

The GRU determines the final output $\mathbf{h}_t$ using a "Soft Switch" controlled by the Update Gate $\mathbf{Z}_t$.

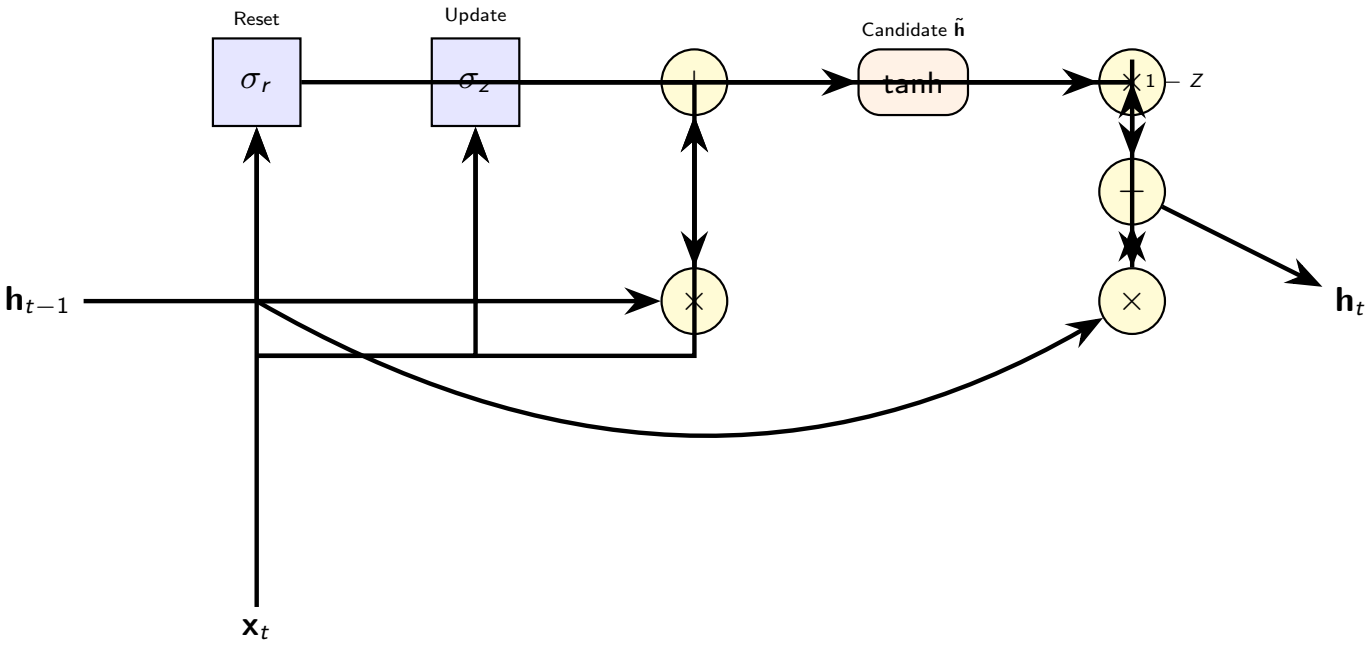
The Interpolation Equation:

$$\mathbf{h}_t = \underbrace{\mathbf{Z}_t \odot \mathbf{h}_{t-1}}_{\text{Keep Old}} + \underbrace{(1 - \mathbf{Z}_t) \odot \tilde{\mathbf{h}}_t}_{\text{Write New}}$$

Intuition:

- **Long-Term Memory ($\mathbf{Z}_t \approx 1$):** The model ignores the current input and copies the old state forward (skipping time steps).
- **Active Processing ($\mathbf{Z}_t \approx 0$):** The model overwrites the state with the new candidate $\tilde{\mathbf{h}}_t$.

5. Visual Summary of the GRU Cell



6. Comparison: RNN vs. LSTM vs. GRU

Feature	Vanilla RNN	LSTM	GRU
Memory Unit	$\mathbf{h}_t$ only	$\mathbf{C}_t$ (Cell) & $\mathbf{h}_t$	$\mathbf{h}_t$ only
Gates	0	3 (In, Forget, Out)	2 (Reset, Update)
Operations	Simple tanh	Linear flow in $\mathbf{C}_t$	Mixing flow in $\mathbf{h}_t$
Parameters	$1\times$ Weights	$4\times$ Weights	$3\times$ Weights
Gradient Flow	Vanishing	Preserved (Identity)	Preserved (Interpolation)

- **RNN:** Fast but forgets long sequences.
- **LSTM:** Powerful, standard for complex dependencies.
- **GRU:** Efficient, comparable performance to LSTM on most tasks.